Relatório de Laboratório

Laboratório de Experimentação de Software — Relatório do Lab03

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab03 — Mineração de Métricas DORA
Grupo (trio)	Leonardo de Souza Galvão · Gabriel Rodrigues Maciel de Abreu · Pedro Henrique Novais Baranda
Link do repositório / GitHub Projects	https://github.com/GabrielRMA1/Lab-experimentacao-medicao-de-software
Data de entrega	22/10/2026

1. Introdução

As métricas DORA (DevOps Research and Assessment) são o padrão de mercado para medir o desempenho de entrega de software: frequência de deploy e lead time for changes medem velocidade; change failure rate e tempo de recuperação medem estabilidade [Forsgren et al. 2018; DORA 2024]. O GitHub, porém, não registra deploys nem falhas em produção. Este laboratório minera proxies públicos — release publicada como deploy, execução de CI com falha como falha — em 342 repositórios open-source populares que usam GitHub Actions, e pergunta também o quanto se pode confiar nesses valores quando a definição operacional muda.

Questões de pesquisa do enunciado e hipóteses informais, formuladas antes da coleta:

- RQ1 — Qual a frequência de deploys dos repositórios populares que usam CI/CD? Hipótese: mediana abaixo de uma release por semana (faixa Medium), pois bibliotecas publicam em ciclos.
- RQ2 — Qual o tempo entre um commit e seu respectivo deploy? Hipótese: lead time por release (a) maior que por commit (b), porque um commit antigo domina a variante (a).
- RQ3 — Qual a taxa de falha das mudanças entregues? Hipótese: CFR de CI (a) maior que o CFR de entrega (b): falhas de pipeline são frequentes e corrigidas antes da release.
- RQ4 — Qual o tempo de recuperação após uma execução de CI/CD com falha? Hipótese: recuperação mediana de poucas horas (menos de 1 dia), pois CI quebrado no branch principal bloqueia o time.
- RQ5 — Repositórios com maior frequência de deploy têm maior ou menor taxa de falha? Hipótese: correlação fraca ou nula, em linha com o DORA (velocidade e estabilidade não são trade-off).
- RQ6 — Quais características estão associadas a melhor desempenho DORA? Hipótese: mais contribuidores, mais releases; diferenças pequenas entre linguagens.
- RQ7 — O quanto a classificação DORA depende da definição operacional? Hipótese: uma parte relevante dos repositórios (mais de 20%) muda de categoria conforme a definição.

Além do enunciado (seção 3.6), o grupo propõe a RQ8 — rework rate, métrica adicionada pelo DORA em 2024 —, duas combinações extras na análise de sensibilidade que mudam uma definição por vez, e uma arquitetura de coleta com cache segmentado e coleta paralela em fatias.

2. Contexto

Este é o Lab03 da disciplina, organizado em três sprints e uma entrega final: a S01 construiu o pipeline de coleta; a S02 previa a amostra completa, as métricas e a validação manual; a S03 faz a análise estatística das RQ1–RQ7. Parte do que estava previsto para a S02 (amostra completa e validação) foi concluída nesta entrega.

O objeto de estudo são repositórios públicos do GitHub com mais de 1.000 estrelas que usam GitHub Actions e publicam releases. A base teórica é o livro Accelerate [Forsgren et al. 2018] e o guia oficial das quatro métricas [DORA 2024]; a classificação Elite/High/Medium/Low usa os cortes fixos adaptados pela disciplina. A análise estatística e as ameaças à validade seguem Wohlin et al. (2012); o Cliff's delta é interpretado segundo Romano et al. (2006) e a correção de comparações múltiplas segue Holm (1979).

3. Metodologia

3.1 Principais Desafios

- Volume e rate limit: os 342 repositórios aceitos somam mais de 2,1 milhões de execuções de CI na janela. Com 5.000 requisições por hora por token, a coleta foi dividida em três fatias, uma por integrante e por token, e os caches foram unidos no final.
- Desempenho do cache: na primeira versão, cada página baixada regravava o arquivo inteiro do repositório. Um repositório muito ativo (openclaw/openclaw) chegou a 185 MB e travou a coleta, com apenas ~100 requisições por hora sendo feitas. O cache foi redesenhado (seção 3.6).
- Tetos da API: a busca devolve no máximo 1.000 repositórios por consulta e a listagem de workflow runs no máximo 1.000 por consulta filtrada; ambos exigiram fatiamento.
- Definições ambíguas: "release corretiva" e "entrega real" exigem julgamento humano, e a heurística automática de release corretiva precisa ser validada.

3.2 Tomadas de Decisão

- Janela de observação de 12 meses: de 02/10/2025 a 02/10/2026
- Commits de cada release obtidos por compare/{release anterior}...{R}, e não por filtro de data: um commit escrito antes da release anterior, mas só incluído agora, continua contando. O custo é uma chamada por release.
- Descarte antecipado, do critério mais barato ao mais caro (Actions → releases → runs → compare), para não gastar cota com repositórios que seriam descartados.
- Amostra final: os aceitos com mais estrelas, com teto de 400 (folga sobre o mínimo de 300 para absorver falhas de coleta). A coleta foi encerrada com 342 aceitos.
- Classificação DORA com métrica ausente: a métrica fica fora da mediana; exigimos ao menos 3 das 4 métricas. Na amostra, todos os 342 repositórios foram classificáveis.
- Estatística: mediana e IQR em todas as RQs; testes não paramétricos (Spearman, Kruskal-Wallis, Mann-Whitney) com correção de Holm e tamanho de efeito; grupos com menos de 5 repositórios ficam fora dos testes.
- Heurística de release corretiva: o dataset foi consolidado com a versão v3_patch_ou_palavras; depois, cinco versões foram avaliadas contra a referência (seção 4.1). A de maior F1 foi v1_patch, mas o dataset não pôde ser reconsolidado com ela (seção 4.1).

3.3 Etapas

O desenvolvimento seguiu em sprints, com as Issues do GitHub Projects abaixo. Cada integrante tem ao menos um commit de código por sprint, referenciando o número da Issue.

Tabela 1. Sprints, entregas e Issues do GitHub Projects.

Sprint	Entregas	Issues
Lab03S01	Seleção de repositórios, metadados e funil; releases, commits entre releases e lead time; workflow runs, cache, retomada, rate limit, CFR e tempo de recuperação; testes e CI	#65, #66, #67
Lab03S02	Coleta completa, dataset e dicionário de dados; commits via compare, heurística corretiva e CFR (b); amostra-ouro e validação	#72, #73, #74
Lab03S03	Correções do pipeline (teto de runs, cache segmentado), classificação DORA,	#75, #76, #77

	análises RQ1–RQ8, relatório	

Responsáveis: cada integrante assumiu uma Issue por sprint, e cada Issue foi fechada por um Pull Request próprio. Pedro Henrique — #67, workflow runs, cache, retomada, rate limit, CFR e tempo de recuperação (PR #70), e #72, coleta, pipeline de repositórios, dataset e dicionário (#75). Leonardo — #66, releases, commits entre releases e lead time (PR #69), e #74, validação, amostra-ouro, kappa de Fleiss e consenso (PR #76). Gabriel — #65, seleção de repositórios, metadados e funil (PR #68), e #73, métricas, commits via compare, heurística corretiva e CFR (PR #77).

Configuração do processo: colunas Backlog → To Do → Doing → Review → Done. Cada Issue tem um responsável (Assignee) e é encerrada por um Pull Request vinculado, revisado antes do merge; todo commit referencia o número da Issue correspondente.

3.4 Ferramentas

- Coleta: API REST do GitHub (versão 2026-03-10), acessada por script próprio em Python 3.12 com a biblioteca requests — sem bibliotecas de terceiros de acesso à API.
- Processamento e estatística: pandas e NumPy; SciPy (Spearman, Kruskal-Wallis, Mann-Whitney, Wilcoxon); statsmodels (kappa de Fleiss, correção de Holm); scikit-learn (precisão/recall/F1, kappa de Cohen ponderado).
- Visualização: Matplotlib (violino, dispersão, mapa de calor e radar).
- Qualidade: pytest e pytest-cov (cobertura de 96% no módulo de métricas, mínimo exigido 80%) e CI no GitHub Actions a cada push.
- Processo: GitHub Projects (v2).

3.5 Tabela de Métricas

Tabela 2. Métricas, definições operacionais e fontes (* = proposta do grupo).

RQ	Métrica	Definição operacional	Unidade	Fonte
RQ1	Deployment frequency	Releases (draft=false, prerelease=false) publicadas na janela ÷ semanas da janela (≈ 52,2)	releases/semana	Script próprio (API REST) — /releases
RQ2 (a)	Lead time por release	Mediana, entre releases R, de published_at(R) − commit.author.date do commit mais antigo de compare/{anterior}...{R}	horas (dias no texto)	Script próprio (API REST) — /compare
RQ2 (b)	Lead time por commit	Mediana, sobre todos os commits de todas as releases, de published_at(R) − commit.author.date	horas (dias no texto)	Script próprio (API REST) — /compare
RQ3 (a)	CFR — proxy de CI	Runs com falha (failure, timed_out, startup_failure) ÷ (falhas + sucessos); push no default branch, criados na janela	%	Script próprio (API REST) — /actions/runs
RQ3 (b)	CFR — proxy de entrega	Releases seguidas em até 7 dias por release corretiva (heurística v3_patch_ou_palavras) ÷ releases avaliadas; últimos 7 dias censurados	%	Script próprio (API REST) + heurística
RQ4	Tempo de recuperação	Por workflow: updated_at do próximo sucesso − run_started_at da 1ª falha após um sucesso; mediana dos episódios; sem sucesso = censurado	horas	Script próprio (API REST) — /actions/runs
RQ5	Velocidade × estabilidade	ρ de Spearman entre RQ1 e RQ3 (a) e (b)	ρ, p, n	SciPy
RQ6	Diferença entre grupos	Kruskal-Wallis ($\varepsilon^2 = H/(n-1)$) por fator; Holm sobre os 16 testes	p, ε^2	SciPy, statsmodels
RQ7	Sensibilidade	% que muda de categoria e kappa de Cohen	%, κ	scikit-learn

		ponderado (linear) entre C1–C4		
RQ8*	Rework rate	Releases corretivas ÷ releases da janela	%	Heurística

4. Resultados

4.1 Coleta de Dados

A busca reuniu 3.000 candidatos com mais de 1.000 estrelas. A coleta avaliou 853 deles, em ordem decrescente de estrelas, e parou ao reunir 342 repositórios que atendem aos critérios do enunciado (Tabela 3). A janela vai de 02/10/2025 a 02/10/2026. Como a coleta começou pelos mais populares, a amostra tem entre 33.216 e 392.645 estrelas.

Tabela 3. Funil de seleção dos repositórios.

Etapa	Repositórios	Descartados	Motivo
Candidatos (busca por estrelas)	3.000	0	—
Avaliados pela coleta	853	2.147	Não avaliados: coleta encerrada ao atingir a cota da amostra
Com GitHub Actions	767	86	Sem workflows (total_count = 0)
≥ 5 releases estáveis na janela	395	372	Menos de 5 releases (draft=false, prerelease=false) na janela
≥ 50 workflow runs válidos na janela	342	53	Menos de 50 runs (push, default branch, success/failure) na janela
Amostra final	342	0	Acima do teto de 400 (maiores por estrelas ficam)

Ao todo foram analisadas 18.292 releases estáveis e 2.188.125 execuções de CI válidas. Dados censurados não foram descartados: 169 dos 55.557 episódios de falha (0,3%) terminaram sem recuperação dentro da janela, e 446 releases dos últimos 7 dias ficaram fora do denominador do CFR (b). Apenas 12 releases foram ignoradas no lead time por compare 404/422, nenhum repositório teve runs truncados pelo teto da API, e 17 repositórios não tiveram nenhum episódio de falha recuperado (sem valor de recuperação). Outliers foram mantidos: todas as estatísticas usam mediana e IQR. Além dos aceitos, 24 repositórios tiveram a coleta interrompida quando a cota foi atingida e ficaram entre os não avaliados.

Validação da heurística de release corretiva

As 300 releases da amostra-ouro (60 repositórios sorteados com random_state = 42, 5 releases de cada) foram rotuladas inicialmente por um modelo de IA (Claude). O modelo leu título, notas e mensagens de commit de cada release e aplicou o guia de rotulagem do grupo, sem acesso à saída da heurística. Em seguida, os três integrantes revisaram os rótulos e concordaram com a classificação. A referência marcou 119 das 300 releases (39,7%) como corretivas. Como a revisão partiu dos rótulos da IA, e não de rotulagens independentes, não há kappa de Fleiss: a Tabela 4 mede a concordância da heurística com essa referência revisada.

Tabela 4. Versões da heurística (n = 300 releases); escolhida: v1_patch.

Versão	Regra	Precisão	Recall	F1
v1_patch	só patch	0,572	0,899	0,699

v2_palavras	≥ 50% commits de correção	0,609	0,328	0,426
v3_patch_ou_palavras	v1 ou v2	0,534	0,933	0,679
v4_patch_e_palavras	patch e algum commit de correção	0,620	0,782	0,691
v5_refinada	patch com sinal, release só de fixes ou notas de hotfix	0,658	0,630	0,644

A avaliação foi feita diretamente sobre o pacote de rotulagem (pacote_rotulagem.json), sem o cache da coleta, que foi perdido antes desta etapa. Duas aproximações decorrem disso: o sinal "versão patch" presume que a release anterior de X.Y.Z é X.Y.(Z−1), e o sinal de commits usa só as até 30 mensagens guardadas no pacote. Os valores da Tabela 4 são, portanto, estimativas.

Nenhuma versão atingiu a meta de F1 ≥ 0,70. A melhor foi v1_patch (F1 = 0,699), seguida de perto pela v4 (F1 = 0,691). A versão usada no dataset, v3_patch_ou_palavras, tem o maior recall (0,933) mas a menor precisão (0,534): marcou 208 das 300 releases como corretivas, contra 119 da referência revisada, com 97 falsos positivos. O padrão de erro é claro: quase toda release corretiva é patch, mas muitas releases patch não são corretivas — projetos que publicam tudo como patch (dependências, documentação, pequenas funcionalidades). Como v1 está contida na v3 (v3 = v1 ou v2), o CFR (b) e a rework rate calculados com a v3 são um limite superior dos que seriam obtidos com a v1. Sem o cache, não foi possível reconsolidar o dataset com a v1; por isso CFR (b), RQ5 (b), RQ7 (C2 e C4) e RQ8 usam a v3 e devem ser lidos como superestimados.

4.2 Visualização Gráfica

Todas as medidas são calculadas por repositório e resumidas pela mediana e pelo intervalo interquartil (IQR), porque as distribuições são muito assimétricas.

RQ1–RQ4: qual o desempenho DORA dos repositórios?

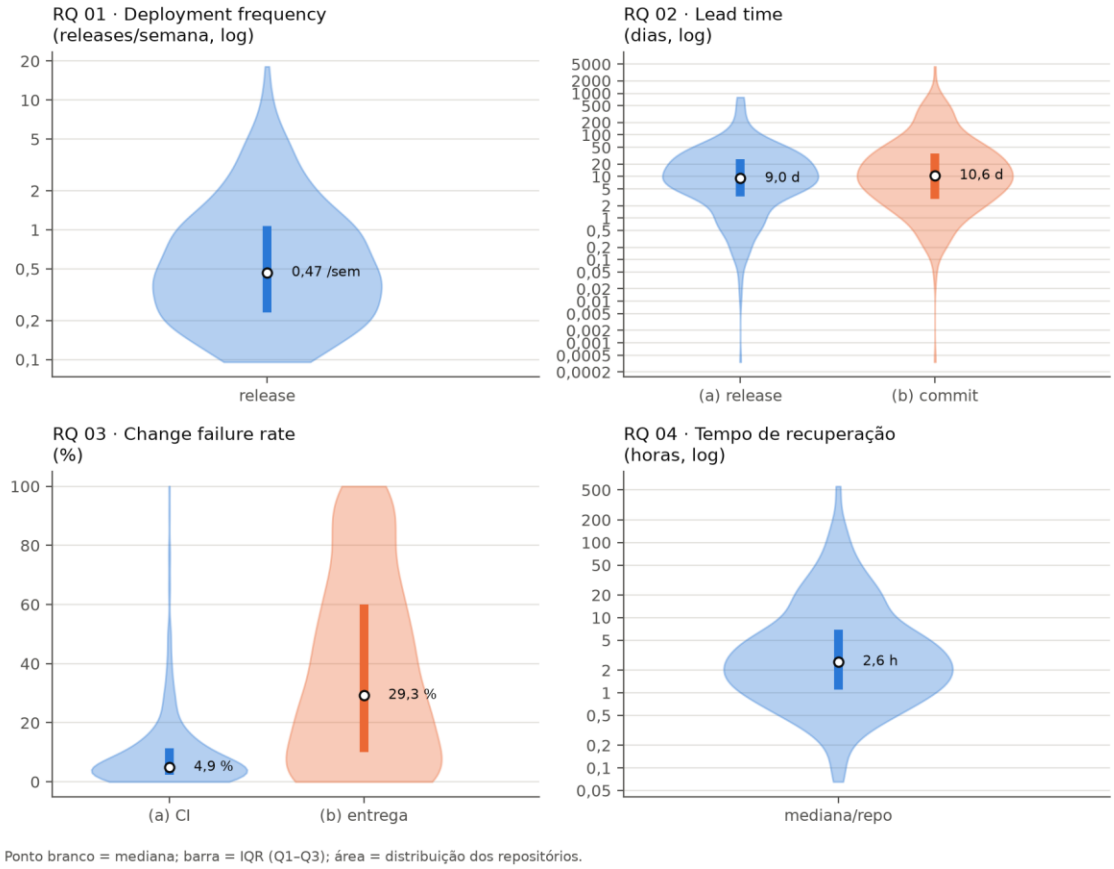


Figura 1. Distribuição das quatro métricas por repositório (violino; ponto = mediana, barra = IQR).

RQ1: a mediana foi de 0,47 (IQR 0,23–1,07) releases por semana — cerca de uma release a cada duas semanas. RQ2: lead time por release (a) de 9,0 dias (IQR 3,3–25,7 dias) e por commit (b) de 10,6 dias (IQR 2,9–35,4 dias). RQ3: CFR de CI (a) de 4,9% (IQR 2,3–11,3%) e CFR de entrega (b) de 29,3% (IQR 10,0–60,0%). RQ4: tempo de recuperação de 2,6 horas (IQR 1,1–6,9 horas).

Tabela 5. Mediana e IQR das métricas por repositório.

RQ	Métrica	Unidade	n	Q1	Mediana	Q3
RQ 01	Deployment frequency	releases/semana	342	0,23	0,47	1,07
RQ 02a	Lead time (a) por release	dias	342	3,3	9,0	25,7
RQ 02b	Lead time (b) por commit	dias	342	2,9	10,6	35,4
RQ 03a	CFR (a) proxy de CI	%	342	2,3	4,9	11,3
RQ 03b	CFR (b) proxy de entrega	%	342	10,0	29,3	60,0
RQ 04	Tempo de recuperação	horas	325	1,1	2,6	6,9
RQ 04	Episódios censurados	% por repo	325	0,0	0,0	0,4

Tabela 6. Classificação DORA na combinação de referência C1 (release, lead time (a), CFR (a)).

Métrica	Elite	High	Medium	Low	Sem nível
Deployment frequency	5 (1%)	86 (25%)	157 (46%)	94 (27%)	0
Lead time (a)	49 (14%)	94 (27%)	124 (36%)	75 (22%)	0
CFR (a)	283 (83%)	37 (11%)	12 (4%)	10 (3%)	0
Recuperação	74 (23%)	219 (67%)	27 (8%)	5 (2%)	17
Geral	18 (5%)	152 (44%)	152 (44%)	20 (6%)	0

Na classificação geral, 152 repositórios (44%) são High, 152 (44%) Medium, 18 Elite e 20 Low. As métricas puxam em direções opostas: o CFR de CI é Elite em 83% dos repositórios e a recuperação é High em 67%, mas a frequência de deploy é Medium ou Low em 73%.

RQ5: repositórios que entregam mais falham mais?

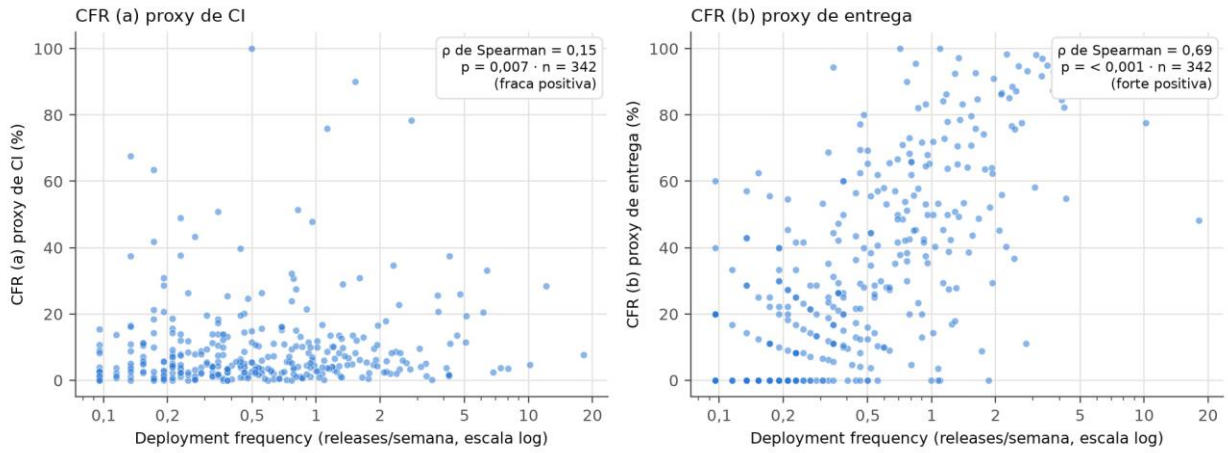


Figura 2. Frequência de deploy (escala log) × CFR, por repositório, com o ρ de Spearman.

CFR de CI: ρ = 0,147 (p = 0,007, n = 342). CFR de entrega: ρ = 0,687 (p = < 0,001, n = 342).

RQ6: quais características estão associadas a melhor desempenho?

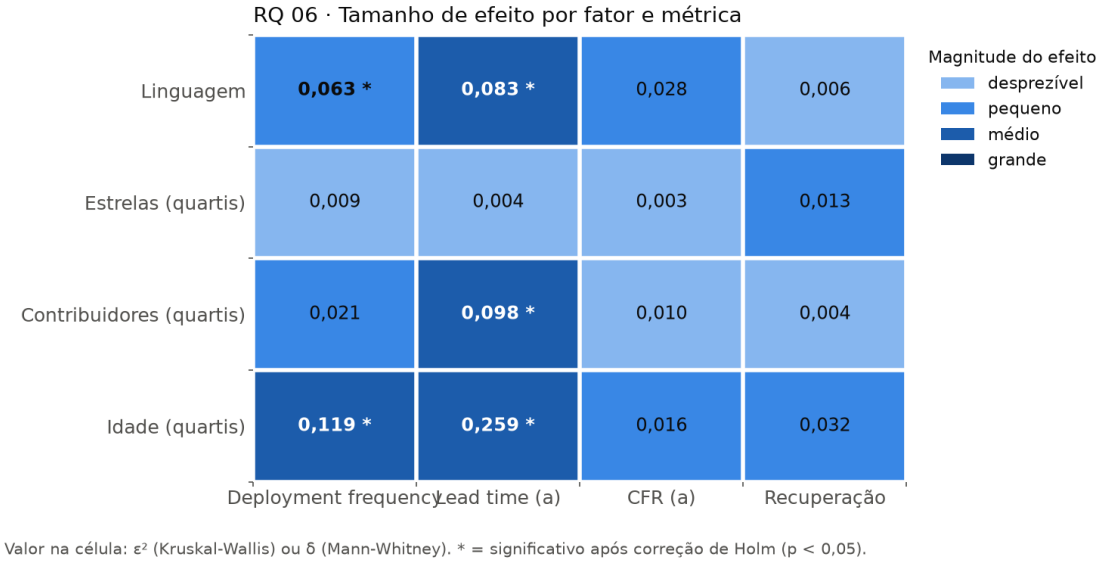


Figura 3. Tamanho de efeito (ε²) de cada fator sobre cada métrica; * = significativo após Holm.

Dos 16 testes, 5 permaneceram significativos após a correção de Holm, todos sobre velocidade (frequência de deploy e lead time); nenhum fator teve efeito significativo sobre o CFR de CI ou o tempo de recuperação. O maior efeito é da idade sobre o lead time (ε² = 0,259, médio): a mediana sobe de 2,3 dias nos repositórios mais jovens (até 2,5 anos) para 20,7 dias nos mais antigos (mais de 10,5 anos), e a frequência de deploy cai de 0,85 para 0,36 releases por semana. Mais contribuidores também se

associam a lead time maior (de 4,8 para 15,3 dias). Entre linguagens, TypeScript e Python entregam mais rápido (lead time de 7,0 e 6,5 dias) que Go (15,2 dias). A popularidade (estrelas) não teve efeito em nenhuma métrica.

RQ7: a classificação muda com a definição?

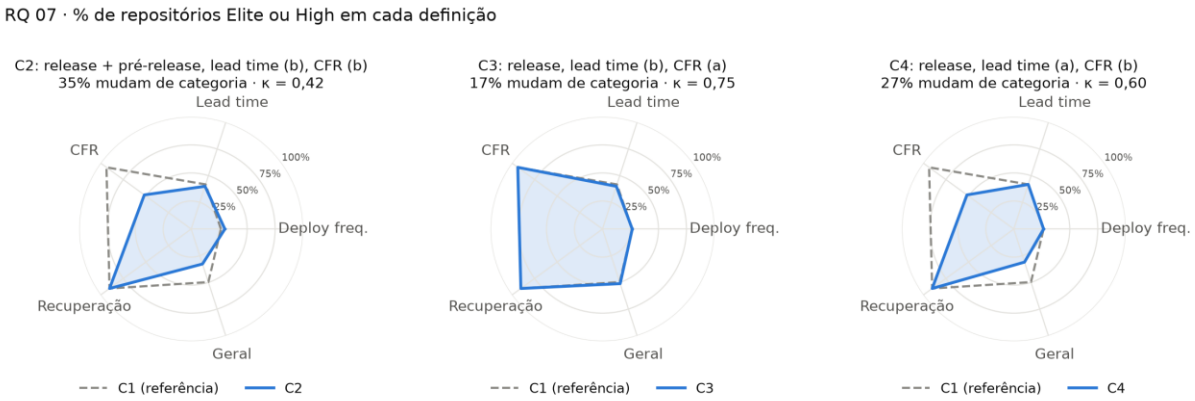


Figura 4. % de repositórios Elite ou High por métrica: cada combinação (azul) sobre a referência C1 (cinza).

Tabela 7. Concordância entre as combinações (n = 342). C1: release, (a), (a); C2: release + pré-release, (b), (b); C3: release, (b), (a); C4: release, (a), (b).

Par	Mudaram de categoria	Kappa ponderado
C1 × C2	35,4%	0,421
C1 × C3	17,3%	0,750
C1 × C4	26,6%	0,596
C2 × C3	28,4%	0,554
C2 × C4	17,0%	0,685
C3 × C4	29,5%	0,536

4.3 Discussão

Hipóteses × resultados

- RQ1 — confirmada. A mediana de 0,47 release por semana fica na faixa Medium, e 73% dos repositórios são Medium ou Low em frequência de deploy. Uma release estável publicada não é um deploy contínuo: muitos projetos open-source entregam por ciclos, e o proxy subestima quem publica de outra forma (imagens, branches).
- RQ2 — refutada no agregado, confirmada em parte. A hipótese era lead time (a) > (b); a mediana geral foi o contrário (9,0 contra 10,6 dias). Ainda assim, (a) ≥ (b) em 56% dos repositórios. A variante (b) pesa cada commit, portanto dá mais peso às releases grandes, que acumulam commits por mais tempo; a (a) dá o mesmo peso a cada release. A escolha muda a resposta — o mesmo achado que a RQ7 quantifica.
- RQ3 — confirmada. O CFR de CI (4,9%) é muito menor que o de entrega (29,3%). Surpreendeu o quão baixo é o (a): 83% dos repositórios seriam Elite. Runs de push no branch principal ocorrem depois do merge, quando o PR já passou no CI; falhas reais ficam nos PRs, que o proxy não vê. O (b), por outro lado, está inflado pela baixa precisão da heurística (seção 4.1).

- RQ4 — confirmada. A recuperação mediana foi de 2,6 horas, e só 0,3% dos episódios ficaram censurados. Parte dessa rapidez provavelmente vem de reexecuções de testes instáveis (flaky), que "recuperam" sem correção alguma.
- RQ5 — parcialmente refutada. Com o CFR de CI a correlação é fraca (ρ = 0,147), compatível com o DORA. Com o CFR de entrega, ρ = 0,687 é forte, mas tende a ser artefato: quem publica toda semana quase sempre tem outra release patch em até 7 dias, e a v3 conta essas patches como corretivas. A janela de 7 dias acopla as duas métricas por construção.
- RQ6 — refutada em parte. Contribuidores não aumentam a frequência de deploy (sem efeito) e se associam a lead time maior; idade é o fator mais forte, com efeito médio sobre o lead time; as linguagens diferem mais do que esperávamos. Nenhum fator explica estabilidade (CFR de CI e recuperação). Estrelas não tiveram efeito, mas a amostra cobre só os muito populares (33.216 a 392.645 estrelas).
- RQ7 — confirmada. Entre C1 e C2, 35,4% dos repositórios mudam de categoria (κ = 0,42, concordância moderada). As combinações de uma definição por vez mostram de onde vem a mudança: só o CFR (C1 × C4) muda 26,6%, só o lead time (C1 × C3) muda 17,3%. A definição de falha pesa mais que a de lead time.

Ameaças à validade

- Construto: release não é deploy, e run de CI com falha não é falha em produção. A heurística de release corretiva ficou abaixo da meta (F1 < 0,70) e foi validada contra rótulos gerados por IA e revisados pelo grupo; a referência pode errar, sobretudo em releases com notas genéricas (“See CHANGELOG.md”), em que só as mensagens de commit estavam disponíveis.
- Interna: o cache foi perdido antes da validação; a avaliação da heurística usa aproximações (seção 4.1) e o dataset permaneceu com a v3, mais permissiva que a melhor versão. Recuperação pode refletir reexecuções.
- Externa: só repositórios muito populares, que usam GitHub Actions e publicam releases; a coleta parou nos 342 primeiros aceitos por estrelas. Os resultados não se estendem a projetos menores ou corporativos.
- Conclusão: 16 testes na RQ6 com correção de Holm; n grande torna significativos efeitos pequenos, por isso reportamos ε^2 e Cliff's delta. Correlação não implica causa (ex.: idade e lead time).

Contribuição das inovações

O cache segmentado e a coleta em fatias foram o que tornou a coleta viável depois do travamento; a subdivisão adaptativa garantiu que nenhum repositório tivesse runs truncados. As combinações C3 e C4 transformaram o "muda de categoria" da RQ7 em uma resposta atribuível: é a definição de falha que mais desloca a classificação.

5. Conclusão

Os 342 repositórios populares analisados entregam em ritmo moderado (mediana de 0,47 release por semana), com lead time de 9 a 11 dias conforme a definição, e se recuperam de falhas de CI em poucas horas. Na combinação de referência, 50% são High ou Elite, puxados pelas métricas de estabilidade, enquanto a frequência de deploy os puxa para baixo.

A principal lição é metodológica: com proxies públicos, a definição operacional decide boa parte do resultado. Um em cada três repositórios (35,4%) muda de categoria entre C1 e C2, e o CFR de entrega depende de uma heurística que, mesmo na melhor versão, não chegou a F1 de 0,70. Classificações DORA mineradas do GitHub devem ser publicadas junto com a definição usada e uma análise de sensibilidade.

Trabalhos futuros: rotulagem humana independente da amostra-ouro (três integrantes e kappa de Fleiss) para confirmar a referência revisada e validar a heurística; reconsolidar o dataset com a melhor versão; usar runs de pull request e eventos de deployment do GitHub como proxies alternativos; e replicar com repositórios menos populares.

6. Referências

DORA. DORA's software delivery metrics: the four keys. 2024. Disponível em: https://dora.dev/guides/dora-metrics-four-keys/.

DORA. A history of DORA's software delivery metrics. Disponível em: https://dora.dev/insights/dora-metrics-history/.

FORSGREN, N.; HUMBLE, J.; KIM, G. Accelerate: The Science of Lean Software and DevOps. IT Revolution, 2018.

HOLM, S. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics, v. 6, n. 2, p. 65–70, 1979.

LANDIS, J. R.; KOCH, G. G. The measurement of observer agreement for categorical data. Biometrics, v. 33, n. 1, p. 159–174, 1977.

ROMANO, J. et al. Appropriate statistics for ordinal level data. Annual Meeting of the Florida Association of Institutional Research, 2006.

WOHLIN, C. et al. Experimentation in Software Engineering. Springer, 2012.

ZUSE, H. A framework of software measurement. Walter de Gruyter, 2013.